

CI/CD für Apps mit GitHub Actions

Wie man das Deployment von Databricks Apps über GitHub automatisiert – mit GitHub Actions und Declarative Automation Bundles.

Diese Seite erklärt, wie man Databricks Apps automatisch aus GitHub heraus bereitstellt. Das ist besonders für Teams relevant, die professionelle Software-Entwicklungsprozesse (CI/CD = Continuous Integration/Continuous Deployment) etablieren möchten, bei denen jede Änderung automatisch getestet und ausgerollt wird.

Voraussetzungen

- Ein Databricks-Service-Principal, dem die bereitgestellte App gehört
- Eine `databricks.yml` -Bundle-Konfiguration im Repository-Wurzelverzeichnis, die die App als Ressource deklariert
- Die Databricks CLI lokal installiert für Einrichtungsaufgaben

Schritt 1: Workload Identity Federation konfigurieren

Ziel ist es, dass GitHub-Actions-Runner sich bei Databricks mit kurzlebigen OIDC-Tokens authentifizieren können, statt fest gespeicherte Zugangsdaten zu verwenden. Das ist deutlich sicherer, weil keine dauerhaften Secrets im Repository liegen.

1. Der Anleitung zur Einrichtung der Workload Identity Federation folgen, um eine GitHub-Actions-Federation-Richtlinie für das Service Principal zu erstellen
2. Die Anwendungs-ID (UUID) des Service Principals sowie die Workspace-URL notieren – beide werden später als Workflow-Variablen benötigt
3. Dem Service Principal die Berechtigung `CAN MANAGE` auf die App geben, bzw. Workspace-Berechtigungen zum Erstellen von Apps, falls nötig

Schritt 2: GitHub-Repository konfigurieren

Unter "Settings" > "Environments" eine neue Umgebung mit dem Namen `prod` anlegen (oder ein anderer Name, der im Workflow referenziert wird). Dort folgende

Umgebungsvariablen hinzufügen:

- `DATABRICKS_HOST` : die Workspace-URL (z. B. `https://my-workspace.cloud.databricks.com`)
- `DATABRICKS_CLIENT_ID` : die Anwendungs-ID des Service Principals aus Schritt 1

Sicherheitshinweis: Diese Werte sind keine geheimen Zugangsdaten. Die Federation-Richtlinie steuert den Zugriff; die Client-ID allein ermöglicht keine Authentifizierung. Ein Client-Secret wird nicht benötigt.

Schritt 3: Bundle für Produktions-Deployments konfigurieren

In der Datei `databricks.yml` sollten für das Ziel (Target) `prod` explizit die Workspace-Adresse (`host`) und der Basis-Pfad (`root_path`) angegeben werden, um sicherzustellen, dass Deployments immer an derselben Stelle landen. Die Produktions-Validierung erfordert beide Angaben, außer wenn `run_as` ein Service Principal festlegt.

Option A: Git-Quelle (empfohlen)

```
targets:
  prod:
    mode: production
    workspace:
      host: https://my-workspace.cloud.databricks.com
      root_path: /Workspace/Users/<service-principal-or-owner>/bundle/${bundle.name}
    resources:
      apps:
        my_app:
          name: my-app
          git_repository:
            url: https://github.com/org/repo
          git_source:
            branch: main
            source_code_path: apps/my-app
```

Vorteil: Der Code wird beim Deployment direkt aus Git geladen, zusätzliche Sync-Schritte entfallen und der bereitgestellte Code bleibt automatisch mit dem Repository synchron.

Bei privaten Repositories muss zuvor ein Git-Zugangsdatum für das Service Principal der App konfiguriert werden. Der Parameter `source_code_path` kann weggelassen werden,

wenn sich der App-Code im Wurzelverzeichnis des Repositorys befindet.

Option B: Workspace-Quelle

```
targets:
  prod:
    mode: production
    workspace:
      host: https://my-workspace.cloud.databricks.com
      root_path: /Workspace/Users/<service-principal-or-owner>/bundle/${bundle.name}
    resources:
      apps:
        my_app:
          name: my-app
          source_code_path: ./app
```

Hierbei werden die App-Dateien aus dem lokalen Repository während des Deployments in den Workspace hochgeladen. `./app` ist durch den tatsächlichen Pfad relativ zur `databricks.yml` zu ersetzen.

Schritt 4: Deploy-Workflow hinzufügen

Datei `.github/workflows/deploy.yml` anlegen:

```
name: Deploy to Databricks Apps
on:
  workflow_dispatch:
    # Für automatisches Deployment bei jedem Push auf main einkommentieren,
    # sobald der Workflow validiert wurde
    # push:
    #   branches: [main]

permissions:
  id-token: write # erforderlich für OIDC-Federation
  contents: read

jobs:
  deploy:
    name: Deploy
    runs-on: ubuntu-latest
    environment: prod
    env:
      DATABRICKS_AUTH_TYPE: github-oidc
      DATABRICKS_HOST: ${ vars.DATABRICKS_HOST }
      DATABRICKS_CLIENT_ID: ${ vars.DATABRICKS_CLIENT_ID }
```

```

steps:
  - uses: actions/checkout@v4
  - name: Install Databricks CLI
    uses: databricks/setup-cli@main
  - name: Validate bundle
    run: databricks bundle validate --target prod
  - name: Deploy bundle
    run: databricks bundle deploy --target prod
  - name: Start or restart app
    run: databricks bundle run my_app --target prod

```

Wichtige Details

- `my_app` ist durch den Ressourcen-Schlüssel aus `resources.apps` in `databricks.yml` zu ersetzen
- Die Berechtigung `id-token: write` ist zwingend erforderlich, um OIDC-Tokens anzufordern
- Die Action `databricks/setup-cli` übernimmt die Authentifizierung automatisch, sobald `DATABRICKS_AUTH_TYPE=github-oidc` gesetzt ist

Wichtige Warnung: `databricks bundle deploy` aktualisiert nur die Ressourcen, startet die App aber **nicht** neu. Nach jedem Deploy muss zusätzlich `databricks bundle run` ausgeführt werden – sonst liefert die App weiterhin den alten Code aus.

Schritt 5: Auf einen erfolgreichen Health-Check warten

Nach dem Deployment sollte geprüft werden, ob die App tatsächlich gestartet ist:

```

- name: Wait for app to be running
  env:
    APP_NAME: my-app
  run: |
    for i in $(seq 1 20); do
      STATE=$(databricks apps get "$APP_NAME" --output json | jq -r '.app_status.state')
      echo "Attempt $i/20: state=$STATE"
      if [ "$STATE" = "RUNNING" ]; then
        exit 0
      fi
      sleep 15
    done
    echo "App did not reach RUNNING state within 5 minutes" >&2
    exit 1

```

Begründung: `databricks bundle run` signalisiert der App nur den Startbefehl, wartet aber nicht auf die tatsächliche Ausführung. Der Start kann z. B. wegen fehlender Abhängigkeiten, fehlender Umgebungsvariablen oder Port-Konflikten fehlschlagen. Diese Warteschleife verhindert, dass die CI-Pipeline "grün" meldet, obwohl die App im Hintergrund abstürzt.

`APP_NAME` muss dem Wert entsprechen, der in `databricks.yml` unter `resources.apps.<key>.name` deklariert ist – nicht dem Bundle-Ressourcen-Schlüssel.

Umgang mit bereits existierenden Apps

Wenn ein anderes Bundle oder eine manuell erstellte App bereits denselben Namen besitzt, verhindert eine Bindung des Bundles an die bestehende App einen Deployment-Fehler:

```
databricks bundle deployment bind my_app <existing-app-name> --target prod --auto-a
```

`my_app` ist der Bundle-Ressourcen-Schlüssel, `<existing-app-name>` der tatsächliche Name der App.

Wichtig: Falls die bestehende App serverseitige Konfiguration enthält (z. B. `budget_policy_id`), die nicht in `databricks.yml` steht, muss diese vor dem erneuten Deployment in die Bundle-Datei übernommen werden. Andernfalls können Terraform-Fehler wie "inconsistent result" auftreten.

Den richtigen Auslöser (Trigger) wählen

Zu Beginn: Mit `workflow_dispatch` starten, also manuellem Auslösen, und mehrere erfolgreiche Durchläufe validieren.

Danach: Sobald der Workflow stabil läuft, `push: branches: [main]` ergänzen, um automatisch bei jedem Merge auf main bereitzustellen.

Sicherheitsnetz: Für die Umgebung `prod` lassen sich unter "Settings" > "Environments" > "prod" > "Deployment protection rules" verpflichtende Reviewer festlegen. Workflow-Läufe warten dann auf eine Freigabe, bevor der Deploy-Job startet.

Die wichtigsten Punkte zusammengefasst

1. **Authentifizierung:** Workload Identity Federation macht gespeicherte Zugangsdaten im Repository überflüssig
2. **Bundle-Ziele:** Der Produktionsmodus erfordert explizite Angabe von `host` und `root_path`
3. **Git vs. Workspace:** Die Git-Quelle hält den Code automatisch synchron, die Workspace-Quelle erfordert manuelle Uploads
4. **Verifikation nach dem Deployment:** Health-Checks verhindern stille Startfehler
5. **Bestehende Apps:** Eine Bundle-Bindung verhindert Namenskonflikte mit zuvor erstellten Ressourcen
6. **Schrittweise Einführung:** Erst manuelle Trigger, danach Automatisierung nach erfolgreicher Validierung

Quelle: <https://docs.databricks.com/aws/en/dev-tools/databricks-apps/cicd-github-actions>
Stand: 2026-08-06